HTML Project Documentation

University Survival Hub

Complete Project Specification — HTML Fundamentals Project

Project Type: HTML-only, multi-page website

Difficulty: Beginner → Intermediate

Estimated Time: 5–7 hours

Primary Focus: Semantic HTML, website structure, navigation, forms, SEO, and clean organization

Styling: ❌ No CSS

Functionality: ❌ No JavaScript

Backend/Database: ❌ Not included

---

1. Project Overview

Project Name

University Survival Hub

Project Idea

University Survival Hub is a practical information website designed to help university students navigate common academic and campus-related challenges.

The website will provide students with useful resources, guides, articles, project information, and a way to contact the website owner.

The project is intentionally built using HTML only.

The goal is not to make the website visually attractive yet. The goal is to create a well-structured semantic HTML foundation that can later be styled with CSS

and enhanced with JavaScript.

---

2. Main Project Goal

The primary goal is to demonstrate that you can take a real-world website idea and convert it into a proper multi-page HTML structure.

By completing this project, you should be able to independently create:

- A multi-page website
- A consistent navigation system
- Semantic page layouts
- Proper headings and content hierarchy
- Lists and tables
- Images with meaningful alt text
- Internal and external links
- Forms
- SEO metadata
- Proper document structure
- Organized HTML files and folders

---

3. Project Scope

Included

The website will contain:

- Home
- Resources
- Projects
- Articles
- Contact

Every page should have a consistent:

- Header

- Navigation
- Main Content
- Footer

Some pages may additionally contain:

- Aside
- Sections
- Articles
- Tables
- Lists
- Forms

Not Included

This project will not include:

- CSS styling
- Responsive CSS
- JavaScript functionality
- DOM manipulation
- Backend
- Database
- Login/signup
- Authentication
- Real form submission
- API integration
- Advanced animations
- Frameworks such as React
- Bootstrap/Tailwind

These can be added in future projects.

---

4. Website Architecture

The basic structure will be:

```
University Survival Hub
│
├── Home
│
├── Resources
│
├── Projects
│
├── Articles
│
└── Contact
```

Every page should be accessible through the main navigation.

Example:

```
Home | Resources | Projects | Articles | Contact
```

The navigation should work from every page.

---

5. Recommended Folder Structure

```
university-survival-hub/
│
├── index.html
├── resources.html
├── projects.html
├── articles.html
├── contact.html
│
├── images/
│   ├── campus.jpg
│   ├── study.jpg
│   └── ...
```

```
│
└── ...
```

The exact number of images and folders can be decided during implementation.

The important goal is to maintain a clean and understandable project structure.

---

6. Common Structure of Every Page

Every page should have a valid HTML document structure:

```
DOCTYPE
html
├── head
│   ├── charset
│   ├── viewport
│   ├── title
│   ├── meta description
│   └── favicon
│
└── body
    ├── header
    │   └── nav
    │
    ├── main
    │
    └── footer
```

The exact content inside each area will vary by page.

---

7. Page Requirements

7.1 Home Page

File: `index.html`

Purpose

Introduce the University Survival Hub and provide students with an overview of what the website offers.

Suggested Structure

```
Header
└── Navigation

Main
├── Hero / Introduction Section
├── What This Website Offers
├── Featured Resources
├── Featured Articles
└── Quick Student Tips

Footer
```

Concepts to Practice

- header
- nav
- main
- section
- headings
- paragraphs
- links
- lists
- images
- article
- footer

The homepage should contain links directing users toward the other sections of the website.

7.2 Resources Page

File: `resources.html`

Purpose

Provide useful resources for university students.

Possible categories:

- Study Resources
- Programming Resources
- Productivity Resources
- University Resources
- Career Resources
- Useful Websites

Suggested Structure

```
Header
└── Navigation

Main
├── Resources Introduction
│
├── Study Resources
│   └── Resource items
│
├── Programming Resources
│   └── Resource items
│
├── Career Resources
│   └── Resource items
│
└── Useful Websites

Footer
```

Concepts to Practice

- section
- article
- headings
- paragraphs

- unordered lists
- ordered lists
- links
- external links
- meaningful link text

7.3 Projects Page

File: `projects.html`

Purpose

Show useful university/project-related information.

Instead of making this a generic portfolio, the content should remain connected to the University Survival Hub concept.

Possible content:

- Academic project ideas
- Recommended project workflow
- Common project mistakes
- Project documentation checklist
- Example project categories

Suggested Structure

```
Header
└── Navigation

Main
├── Projects Introduction
├── Project Ideas
├── Project Planning Guide
├── Project Checklist
└── Common Mistakes

Aside
└── Quick Project Tips
```

```
Footer
```

Concepts to Practice

- section
- article
- aside
- lists
- headings
- links
- paragraphs

7.4 Articles Page

File: `articles.html`

Purpose

Provide educational and practical articles for students.

Possible articles:

- How to survive your first semester
- How to manage university assignments
- How to start programming
- How to prepare for exams
- Common mistakes university students make

Each article should have a meaningful structure.

Example:

```
Article
├── Heading
├── Introduction
├── Section
├── Section
└── Conclusion
```

Concepts to Practice

- article
- section
- heading hierarchy
- paragraphs
- lists
- links
- semantic structure

7.5 Contact Page

File: `contact.html`

Purpose

Allow visitors to provide their contact information and send a message.

Required Form Fields

At minimum:

- Name
- Email
- Subject
- Message

Possible additional fields:

- Category
- Student/Visitor type
- Preferred contact method

Structure

```
Header
└── Navigation

Main
├── Contact Introduction
```

```
├── Contact Information
└── Contact Form
    ├── Name
    ├── Email
    ├── Subject
    ├── Message
    └── Submit Button

Footer
```

Concepts to Practice

- form
- label
- input
- textarea
- select
- option
- button
- required
- fieldset
- legend
- name
- id
- for
- input types

---

8. Semantic HTML Requirements

One of the most important goals of this project is semantic structure.

Use an appropriate semantic element when one exists.

Expected Elements

Element	Expected Usage
`<header>`	Page/site introductory area
`<nav>`	Main navigation
`<main>`	Primary page content
`<section>`	Logical content sections
`<article>`	Independent content/article
`<aside>`	Related/supporting information
`<footer>`	Footer information
`<div>`	Generic container when no semantic element fits
`<span>`	Inline grouping when needed

Important Rule

Do not use `<div>` for everything.

The HTML should communicate the meaning and structure of the content, not just create containers.

---

9. Navigation Requirements

The navigation bar must appear on all five pages.

It must contain working links to:

- Home
- Resources
- Projects
- Articles
- Contact

The links must use correct relative paths.

Example conceptually:

```
index.html
resources.html
projects.html
```

```
articles.html
contact.html
```

Navigation Checklist

- [ ] Navigation exists on every page
- [ ] All five pages are linked
- [ ] Links work from every page
- [ ] No broken internal links
- [ ] Link text is meaningful
- [ ] Navigation is placed inside `<nav>`

---

10. SEO Requirements

Each page must contain appropriate basic SEO metadata.

Required

- charset
- viewport
- title
- meta description
- lang

Each page should have its own meaningful `<title>`.

For example:

```
University Survival Hub
University Resources | University Survival Hub
Student Projects | University Survival Hub
Student Articles | University Survival Hub
Contact | University Survival Hub
```

The meta description should also be relevant to the individual page.

SEO Principles

- One clear main topic per page
- Meaningful page title
- Proper heading hierarchy
- Descriptive links
- Meaningful image alt text
- Correct language declaration
- Useful meta description

Advanced SEO techniques are not required.

---

11. Images

Images should be used where they actually contribute to the content.

Possible examples:

- University campus
- Students studying
- Books
- Programming
- Projects

Every meaningful image should have appropriate alt text.

Example concept:

```
Image → describes what the image represents
```

Do not use meaningless alt text such as:

- image
- photo
- picture

Decorative images can use an empty alt where appropriate.

---

12. Lists

The project should provide opportunities to practice both major list types.

Unordered Lists

Useful for:

- Resources
- Tips
- Features
- Categories
- Checklists

Ordered Lists

Useful for:

- Step-by-step guides
- Study processes
- Project workflows
- Instructions

Nested lists may be used where they make structural sense.

---

13. Tables

At least one meaningful table should be included somewhere in the website.

For example:

Project Planning Table

Stage	Purpose	Suggested Action
Planning	Define the idea	Write requirements
Research	Understand the topic	Collect information
Development	Build the project	Implement
Testing	Find problems	Test functionality
Submission	Finalize	Review and submit

The table should contain actual structured data, not be used for page layout.

---

14. Links

The website should demonstrate different types of links.

Internal Links

Navigation between pages.

External Links

Useful external resources.

Same-page Links

At least one page may use section links such as:

```
#study-resources
#career-resources
```

Other Useful Links

Where appropriate:

- Email
- Telephone
- External educational resources

---

15. Forms & Validation

The contact form should use the form concepts already learned.

At minimum:

- Name → text
- Email → email
- Subject → text
- Message → textarea
- Submit → submit button

Use appropriate validation such as:

- required

- appropriate input types
- useful placeholders where appropriate

The form does not need to actually send data because there is no backend.

The objective is to correctly structure the form.

---

16. Accessibility Considerations

Accessibility should be incorporated into the HTML structure.

Requirements

- Proper heading hierarchy
- Labels connected to form controls
- Meaningful link text
- Meaningful image alt text
- Correct semantic elements
- lang attribute
- Do not rely on visual styling to communicate meaning

You do not need to implement advanced accessibility techniques for this project.

---

17. HTML Concepts Covered

This project acts as a final integration project for your HTML fundamentals.

Fundamentals

- HTML document structure
- Elements
- Nesting
- Attributes
- Headings
- Paragraphs
- Comments

- Text formatting

Navigation & Media

- Links
- Relative paths
- Images
- File paths
- Audio/video/iframe awareness
- Responsive images awareness

Not every previously learned media concept needs to appear just for the sake of using it.

Structure & Semantics

- Block vs inline
- div
- span
- Classes
- IDs
- Semantic HTML
- Header
- Navigation
- Main
- Section
- Article
- Aside
- Footer

Data

- Lists
- Nested lists

- Tables

Document Head

- `<title>`
- `<meta charset>`
- viewport
- meta description
- favicon
- lang

Forms

- form
- label
- input
- input types
- textarea
- select
- option
- button
- fieldset
- legend
- validation attributes

DOM

The project should not require JavaScript DOM manipulation.

You already learned the basic idea of the DOM; actual DOM programming belongs to the JavaScript phase.

---

18. Project Development Phases

Phase 1 — Planning

Estimated time: 30–45 minutes

Tasks:

- Understand requirements
- Decide page content
- Plan navigation
- Plan semantic structure
- Decide where lists/tables/forms/images will be used
- Decide folder structure

Output: Website blueprint.

Phase 2 — Basic Page Skeleton

Estimated time: 30–45 minutes

Create all five HTML files.

Establish:

- DOCTYPE
- html
- head
- body
- header
- nav
- main
- footer

Do not focus on detailed content yet.

Phase 3 — Homepage

Estimated time: 45–60 minutes

Build the homepage with:

- Introduction
- Sections

- Links
- Image
- Featured content

Phase 4 — Resources + Projects

Estimated time: 60–90 minutes

Build:

- Resources page
- Projects page

Practice:

- sections
- articles
- lists
- links
- aside
- table

Phase 5 — Articles

Estimated time: 45–60 minutes

Create several meaningful article previews or articles.

Focus heavily on:

- heading hierarchy
- article
- section
- paragraphs
- lists
- semantic structure

Phase 6 — Contact Form

Estimated time: 30–45 minutes

Create the contact page and form.

Practice:

- labels
- input types
- textarea
- select
- validation
- buttons
- fieldset/legend

Phase 7 — SEO + Final Cleanup

Estimated time: 30–45 minutes

Review every page.

Check:

- title
- meta description
- viewport
- favicon
- lang
- headings
- alt text
- links
- semantic structure

Phase 8 — Final Testing

Estimated time: 30–45 minutes

Test the website as a user.

Click:

- Home → Resources
- Resources → Projects
- Projects → Articles
- Articles → Contact
- Contact → Home

Check every page for:

- broken links
- incorrect paths
- missing elements
- incorrect nesting
- missing labels
- missing alt text
- missing metadata

---

19. Estimated Total Time

Target: 5–7 hours

This is an estimate, not a deadline.

The project should be completed in a way that prioritizes understanding and independent implementation, not speed-running.

Suggested breakdown

Phase	Time
Planning	30–45 min
Skeleton	30–45 min
Homepage	45–60 min
Resources + Projects	60–90 min
Articles	45–60 min
Contact	30–45 min
SEO/Cleanup	30–45 min
Testing	30–45 min

Phase	Time
Total	**~5–7 hours**

---

20. What You Should NOT Do

To keep the project aligned with its purpose:

❌ Do not add CSS

No style.css, inline styles, or style tags.

❌ Do not add JavaScript

No script tags, click handlers, or DOM manipulation.

❌ Do not use frameworks

No Bootstrap, Tailwind, React, or Vue.

❌ Do not copy a complete website

You may take inspiration from other websites, but the structure and content should be your own.

❌ Do not overcomplicate the website

This is an HTML fundamentals project, not a production application.

---

21. Quality Standards

The project will be judged on structure, not appearance.

Excellent HTML should be:

- Semantic
- Organized
- Readable
- Consistent
- Properly nested
- Meaningful
- Accessible

- Easy to style later
- Free of unnecessary elements

---

22. Final Completion Checklist

Website Structure

- [ ] Five pages created
- [ ] Every page has valid document structure
- [ ] Every page has header
- [ ] Every page has navigation
- [ ] Every page has main content
- [ ] Every page has footer

Navigation

- [ ] Home works
- [ ] Resources works
- [ ] Projects works
- [ ] Articles works
- [ ] Contact works
- [ ] Navigation works from every page

Semantic HTML

- [ ] Correct use of header
- [ ] Correct use of nav
- [ ] Correct use of main
- [ ] Correct use of section
- [ ] Correct use of article
- [ ] Aside used where appropriate
- [ ] Footer used correctly

- [ ] Div used only when necessary

Content

- [ ] Headings are logical
- [ ] Heading hierarchy is correct
- [ ] Paragraphs are meaningful
- [ ] Lists used appropriately
- [ ] At least one meaningful table
- [ ] Images used appropriately
- [ ] Images have proper alt text
- [ ] Links have meaningful text

Forms

- [ ] Contact form exists
- [ ] Name field
- [ ] Email field
- [ ] Subject field
- [ ] Message field
- [ ] Labels correctly associated
- [ ] Appropriate input types
- [ ] Required fields where appropriate
- [ ] Submit button

SEO

- [ ] lang attribute
- [ ] Charset
- [ ] Viewport
- [ ] Unique title on every page
- [ ] Meta description on every page

- [ ] Favicon
- [ ] Logical heading structure
- [ ] Descriptive links
- [ ] Meaningful alt text

Final Testing

- [ ] No broken links
- [ ] No missing pages
- [ ] No incorrect file paths
- [ ] No unnecessary code
- [ ] No CSS
- [ ] No JavaScript
- [ ] HTML is readable and organized

---

23. Learning Outcomes

After completing this project independently, you should be able to:

1. Build a multi-page website

You will understand how separate HTML documents combine to form one website.

2. Think in website structure

Instead of thinking: *"Which tag should I use?"*

you should increasingly think: *"What is the semantic structure of this content?"*

3. Use semantic HTML

You will be able to distinguish between header, nav, main, section, article, aside, footer, and understand why each exists.

4. Build real forms

You will be able to structure a basic real-world contact form independently.

5. Implement basic SEO

You will understand the role of title, meta description, lang, headings, alt, meaningful links.

6. Organize a website project

You will understand basic HTML files, folders, images, relative paths, page relationships.

7. Prepare HTML for CSS

Most importantly, you will learn to create HTML that can later be styled without having to completely rebuild the structure.

---

24. Future Evolution

The project is intentionally designed to evolve.

Stage 1 — Current

```
HTML
├── Structure
├── Content
├── Semantics
├── SEO
├── Forms
└── Navigation
```

↓

Stage 2 — Future

```
CSS
├── Layout
├── Colors
├── Typography
├── Spacing
├── Responsive design
└── Components
```

↓

Stage 3 — Future

```
JavaScript
├── Interactions
├── Dynamic content
├── Form handling
├── DOM manipulation
└── Search/filtering
```

↓

Stage 4 — Future

```
Backend + Database
├── User accounts
├── Real submissions
├── Articles database
├── Resources database
├── Search
└── Admin panel
```

So this isn't just a throwaway HTML exercise.

The same University Survival Hub can eventually become a proper full-stack project.

---

25. Definition of Success

The project is considered successful not when it looks good, but when:

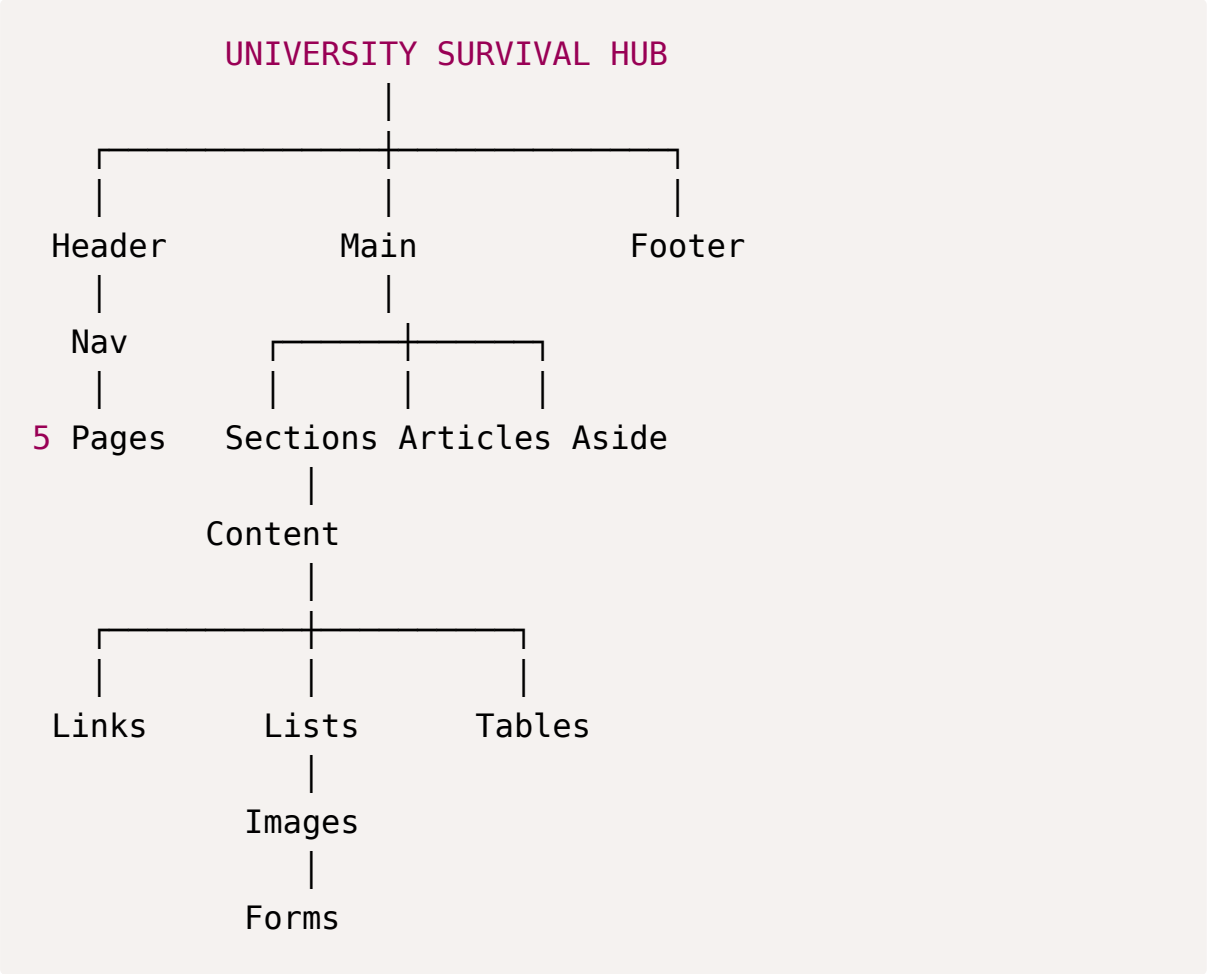
A developer can open your HTML files, understand the entire website structure immediately, navigate between every page, identify the purpose of every major section, and later add CSS/JavaScript without having to rebuild the HTML architecture.

That is the real objective of this project.

---

26. Final Project Target

By the end, you should have something conceptually like:



Project principle:

Structure first. Style later. Functionality after that.

This gives us a proper Lesson 24 capstone project rather than just another HTML exercise.